Welcome.

Everyone:

- Pull the updates from the course GitHub repo:
  - `cd <46120-PiWE repo>`
  - `git pull origin main`

NB:

- By attending this class, you consent to being recorded. Recording will be shared to this class and possibly other DTU students for training purposes.

46120: Scientific Programming for Wind Energy

Packaging, linters and more

Jenni Rinker

Agenda for today

- Pull new course material ✔

- Results from the midterm evaluation
- PEPs and linters
  - (09:30 Presentations from related special courses)
- Packaging

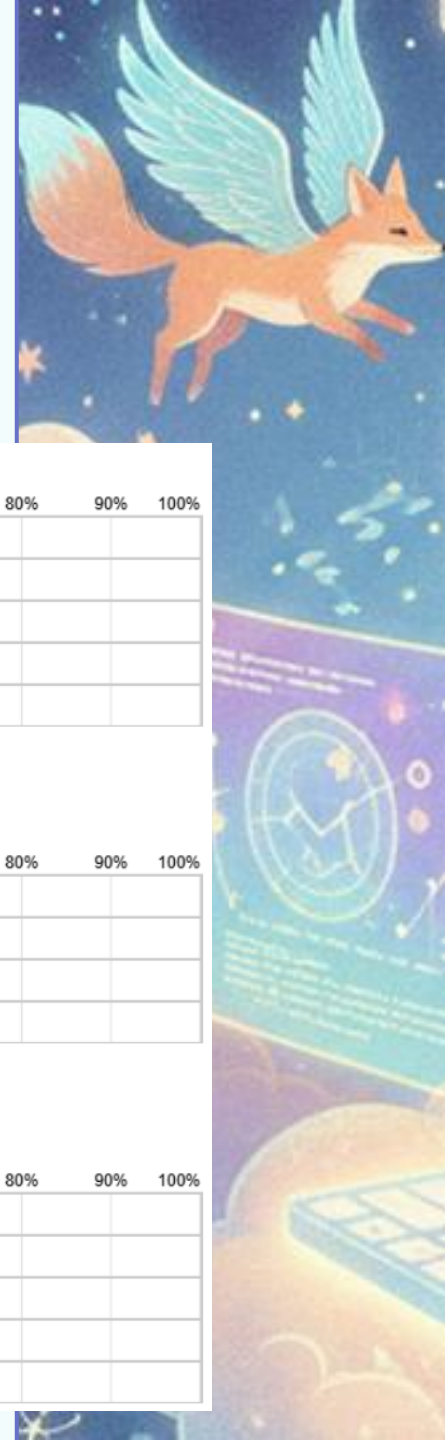
- Begin homework

Midterm evaluation results

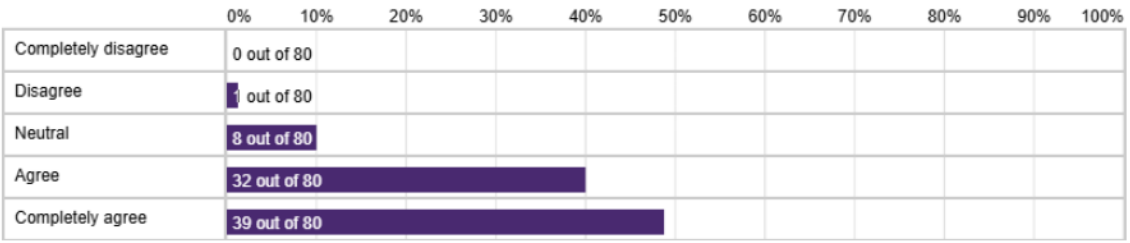
How your feedback has changed the course plan.

- 80/117 (68%), low completion (75% goal).
- Generally very good.
- Small improvements from last year (next slide).
- Some room for improvement on clarity, mentioned in free text.

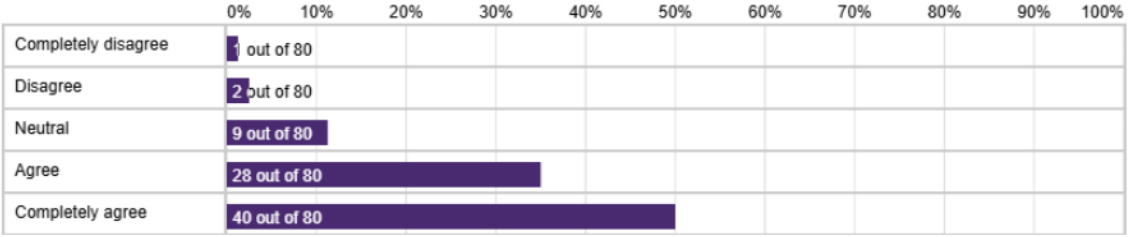
- But looks like students mostly like it. Cool!



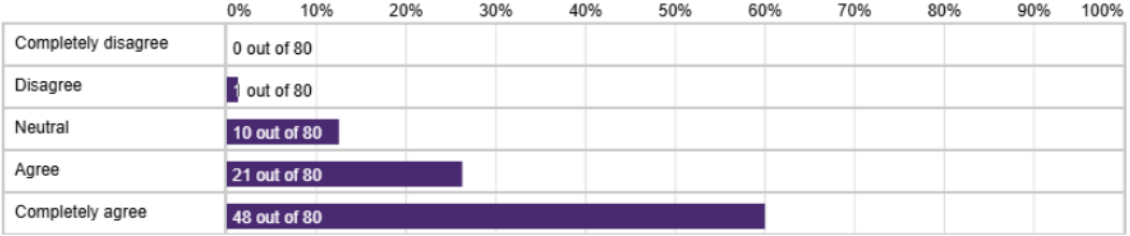
1 I am learning a lot in this course



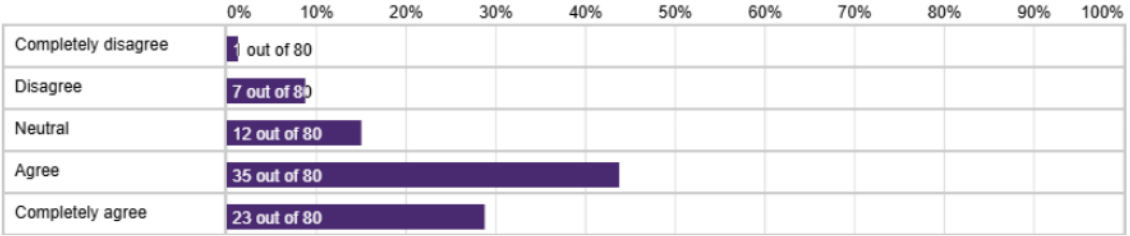
2 The learning activities motivate me to work with the material



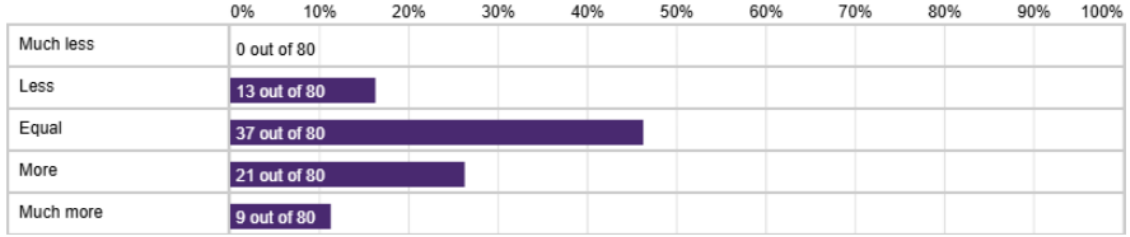
3 During the course, I have had the opportunity to get feedback on my performance



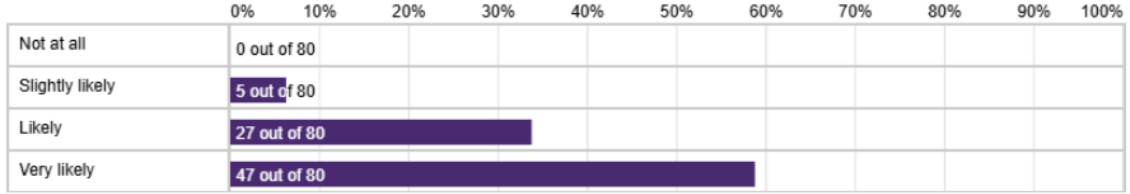
4 It is generally clear what is expected of me in exercises, project work, etc.



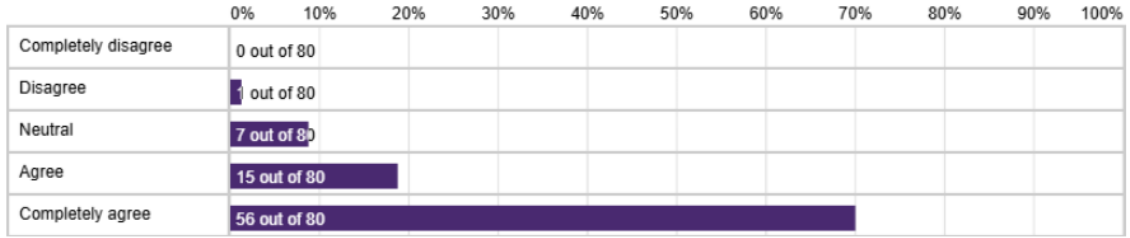
5 5 points corresponds to 9h./week. I think that my average workload in this course is



6 How likely are you to recommend this class to other students or colleagues?

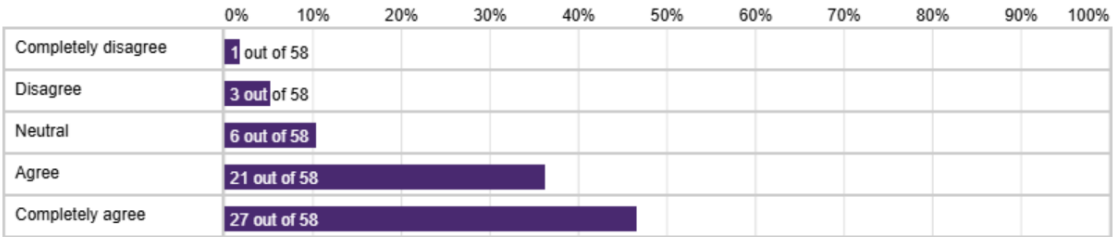


7 I am generally enjoying working with my group on the assignments

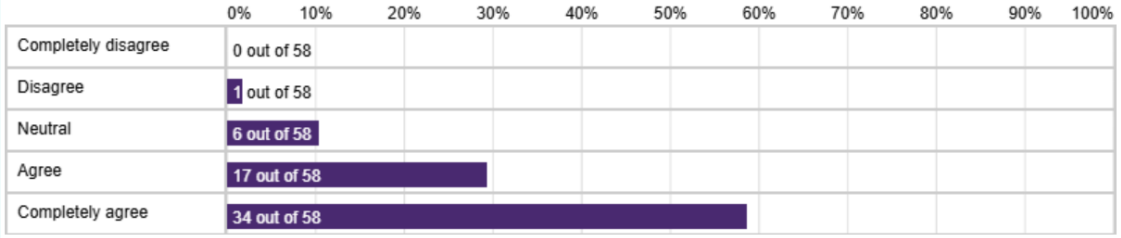


- Results from 2025 (last year)

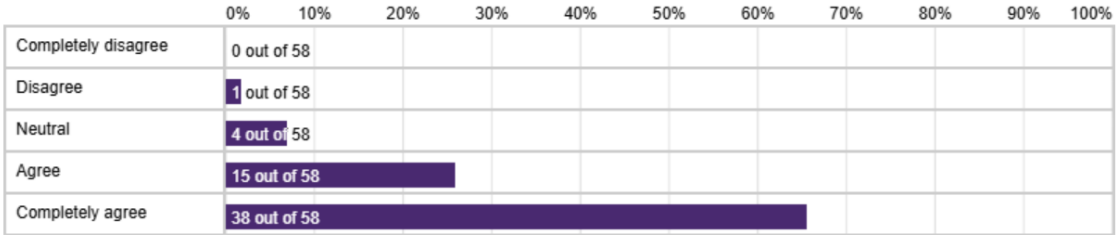
1 In my opinion, I am learning a lot in this course



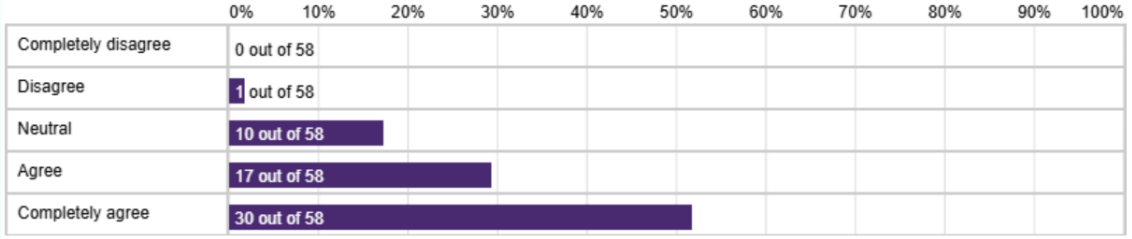
2 The learning activities motivate me to work with the material



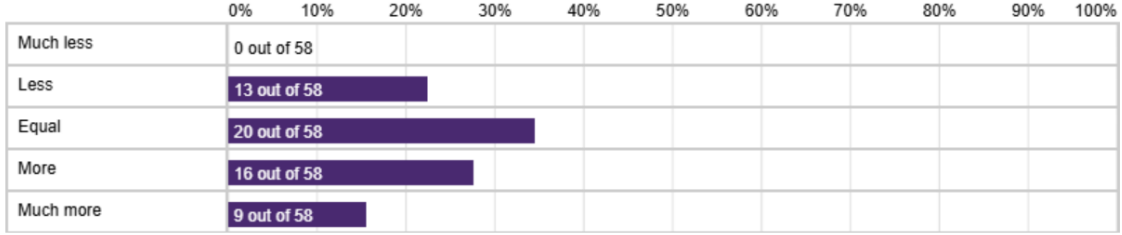
3 During the course, I have had the opportunity to get feedback on my performance



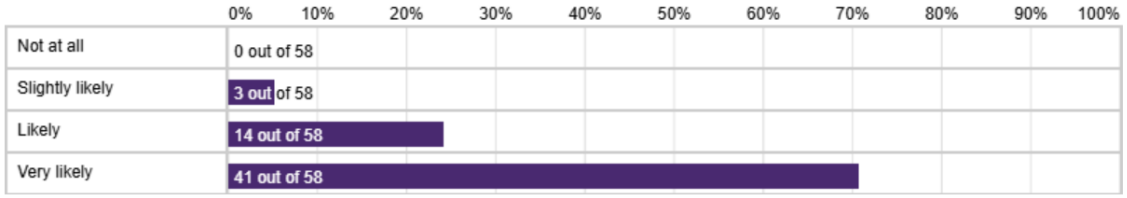
4 It is generally clear what is expected of me in exercises, project work, etc.



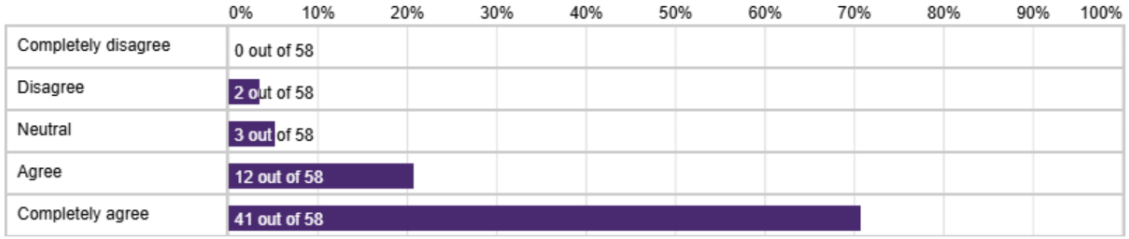
5 5 points corresponds to 9h./week. I think that my average workload in this course is



6 How likely are you to recommend this class to other students?



7 I am generally enjoying working with my group on the assignments



Copilot summary of “What (if anything) did you like about the CodeCamp project?”

1. Clear, well-structured weekly workflow → ⭐⭐⭐⭐⭐ (5/5)
Very frequently mentioned. Students repeatedly praised the week-by-week structure, clear tasks, to-do lists, and fair workload.
2. Strong introduction to Python; hands-on coding confidence → ⭐⭐⭐⭐☆ (4/5)
Often mentioned. Many students felt they learned Python basics, improved their skills, and liked building real functionality.
3. Learning Git/GitHub & real collaboration workflows → ⭐⭐⭐⭐⭐ (5/5)
One of the most repeated themes. Students emphasized branches, commits, PRs, collaborative coding, real-project simulation, and better understanding of Git.
4. Good division of tasks across group members → ⭐⭐⭐☆☆ (3/5)
Mentioned multiple times. Students liked that tasks were split logically and matched group size.
5. Realistic wind-energy problem with meaningful data → ⭐⭐⭐⭐☆ (4/5)
Frequently highlighted. Students liked applying code to real wind-energy data and seeing real-world relevance.
6. Valuable “small-scale real project” learning experience → ⭐⭐⭐⭐☆ (4/5)
Mentioned often. Included comments about functions, testing, architecture, scientific workflow, and multi-person teamwork.
7. Helpful guidance and beginner-friendly teaching → ⭐⭐⭐☆☆ (3/5)
Repeated across responses. Students appreciated the early-week guidance, step-by-step approach, and clear explanations.
8. Motivating, intuitive, and enjoyable to work on → ⭐⭐⭐☆☆ (3/5)
Several mentions. Students described the project as fun, intuitive, not too hard, and confidence-building.

Copilot summary of “What would you like to see improved with the CodeCamp project?”?

1. Need for clearer, centralized instructions — ⭐⭐⭐⭐⭐
Students repeatedly requested **one unified document** with: project overview, physics, rubric, file structure, and final-project expectations. Current instructions felt scattered across Learn, GitHub, PDFs, and READMEs.
2. Week 5–6 confusion & unclear final project — ⭐⭐⭐⭐☆
The shift to the final project felt abrupt. Many were unsure what the final deliverable was, how it differed from homework, and which files to create (e.g., `main.py`).
3. Peer-feedback issues (fairness & timing) — ⭐⭐⭐☆☆
Rubric interpretation varied greatly. The 2-day review window was stressful, and groups found it hard to coordinate.
4. Need more physics background (TURBIE basics) — ⭐⭐⭐☆☆
Several students struggled with simulate_turbie() and wanted a short, clear physics explanation or required pre-reading.
5. Better organization of materials & file structure — ⭐⭐⭐⭐☆
Documents felt scattered. Students want **all project info in one place**, plus clarity on required files and folder structure.
6. Workload/pacing concerns — ⭐⭐☆☆☆
Some felt weekly pace was too fast or workload heavy, especially around deadlines.
7. More upfront overview at project start — ⭐⭐⭐☆☆
Students asked for clearer early explanation of project goals, structure, and how weekly tasks connect.
8. A few felt everything was fine — ⭐☆☆☆☆
A small minority said no improvements needed.

Copilot summary of “Do you have any other general feedback”?

1. Course materials scattered / too many platforms
5 students (~14%)
Students found information split across Learn, GitHub, PiWe repo, CodeCamp repo, Slack → hard to locate what’s needed.
2. Peer feedback disliked / unclear value
3 students (~9%)
Concerns about fairness, usefulness, and desire for TA involvement.
3. Need clearer explanations & more detailed guidance
5 students (~14%)
Students mentioned feeling lost during lectures or unclear about homework/tasks.
4. Beginners need more introductory support
3 students (~9%)
Requests for extra intro lecture, simpler explanations, or slower early pace.
5. Lectures start too late / time could be used better
2 students (~6%)
Students prefer using the full allocated class time.
6. Confusion about task expectations (especially project)
2 students (~6%)
Students unclear about what to do next or what is required.
7. Desire for more live-coding demonstrations
2 students (~6%)
Requests for real-time coding examples during lectures.
8. Positive: strong teaching, structure, collaboration focus
6 students (~17%)
Praise for teaching style, course setup, hybrid format, and learning Git/GitHub in groups.

What we will change based on your feedback

- Feedback is generally very positive.
- Only 1 major change for next year: create single document “all you need to know for the CodeCamp project.”
- Everything else will remain, more or less, “as is”.
  - Talk to me if you want to know why we’re not addressing your feedback. There may be a reason OR I may have missed it.

What happens next?

- Start thinking about groups for the final project.
  - Sign-up deadline is Sunday March 29 at 23:59.
- 2 to 3 people. Can be different than CodeCamp group.
  - Can work alone with permission.
- Final project topics:
  1. Wind resource assessment
  2. BEM
  3. Wind power forecasting with ML
  4. Custom project (requires permission)
- More details on topics next week.
  - Last year's descriptions (to be updated).

Week 7	19-mar	Packaging and dependencies	
Week 8	26-mar	Final project introduction	Sign up for final project group
EASTER	02-apr	--	
Week 9	09-apr	Classes, inheritence and OOP	
Week 10	16-apr	Computational thinking and design of architecture	
Week 11	23-apr	Midway presentations	
Week 12	30-apr	Python on Gbar (HPC)	Final project due Mon. May 4 (pass/fail)
Week 13	07-may	Quiz, final-project presentations	Peer reviews presented in class

Questions?

Special courses related to DigiWind

Further your digitalization knowledge.

Overview

Name	Main responsible	Presented by	When
Wind Power Plant System Engineering and Optimization	Pierre-Elouan Rethore	Pierre-Elouan	Fall, 13-week
HPC4Wind	Hamid Sarlak	(same)	Fall, 13-week
Cyber-physical security in power systems	Licio Romao	(same)	TBD

Wind Power Plant System Engineering and Optimization

- Course responsible(s)
  - Pierre-Elouan, Tuhfe Göçmen, Mike McWilliam
  - pire@dtu.dk
  - https://forms.gle/prHq3v7kcUPGZKgt8
- Course timing
  - Fall 13-week, 5 ECTS
- Assumed pre-requisites
  - Scientific Python programming
  - Wind farm planning
- Assessment method
  - Capstone project and oral presentation
- Lecture schedule

Week	Topic
1	Whole-system kick-off and wind farm game
2	Systems engineering for wind and hybrid plants
3	Optimization foundations
4	PyWake — wake modelling and interpretation
5	TopFarm workflows
6	Electrical cable optimization and OptiWindNet
7	HyDesign and hybrid plant design
8	Wind farm flow control
9	Load surrogates and multi-fidelity workflows
10	Floating wind farm optimization
11	Captsone project work
12	Captsone project work
13	Captsone project presentation

Wind Power Plant System Engineering and Optimization: Learning objectives

By the end of the course, students should be able to:

- Explain how systems-engineering principles shape the design and operation of wind and hybrid power plants.
- Formulate wind-energy design questions as optimization problems with objectives, constraints, variables, and assumptions.
- Use PyWake to model wake interactions, interpret simulation outputs, and connect them to design choices.
- Use TopFarm to set up and solve wind farm optimization problems and compare design alternatives under realistic constraints.
- Use OptiWindNet to design internal electrical network design and reason about cable-routing trade-offs.
- Use HyDesign to assess hybrid renewable plant configurations and compare their technical and economic viability.
- Use coding agents productively but critically for wind and hybrid power plant design and control: specify tasks, inspect generated code, validate results, and document limitations.
- Deliver a capstone project that integrates modelling, optimization, systems thinking, and engineering communication into a justified design recommendation.

Course name: high Per5formance Computing for Wind Energy (HPC4Wind)

- Course responsible(s)
  - Hamid Sarlak (DTU Wind)
  - Bernd Dammann (DTU Compute)
- Course timing
  - Fall 13-week (Timeslot TBA)
- Assumed pre-requisites
  - Basic Programming and Wind Energy knowledge
- Assessment method
  - Reports and oral exam

- Lecture schedule

Week	Topic
1	• What is HPC • Introduction to Scientific Computing in Wind Energy
2	• Python bootcamp • Basics of Unix
3	Memory Hierarchy, Cache, Data Loading Wind Farm Design
4	Parallelism concepts (Amdahl's law, Roofline etc.), multiprocessing, threading, and basics of parallel debugging
5	Advanced parallel concepts
6	Parallel (post)processing OpenFOAM structure and RANS/LES turbine wake physics
7	Profiling tools, performance counters, diagnostics
8	GPU basics, CUDA aware libraries, job arrays
9	Language Interfaces for HPC I FEA & Structural Design of WindTurbine Components
10	Language Interfaces for HPC II
11	I/O Strategies for HPC
12	Best Practices in HPC
13	Feedback & Reporting

HPC4Wind: learning objectives

Students completing the course will be able to:

1. Analyse scientific computing problems in wind energy and implement efficient HPC solutions.
2. Explain performance characteristics of CPUs, GPUs, memory hierarchy, and I/O.
3. Utilize hardware/software synergy in performance critical programs.
4. Choose and use appropriate computational resources for a given problem.
5. Practicing data driven techniques by means of HPC simulations
6. Identify bottlenecks and optimize program efficiency.
7. Apply parallelization techniques.
8. Evaluate scalability of parallel programs.
9. Use analytical performance models to assess resource utilization.

pre-register here!

For more information please email: hsar@dtu.dk

Cyberphysical security of power systems: overview

- Course responsible(s)
  - Licio Romao, licio@dtu.dk
- Course timing
  - August 3-week period or fall 13-week (to be decided)
- Assumed pre-requisites
  - Basic course on linear algebra
  - Basic feedback control
  - Electric circuit
- Assessment method
  - Pass/fail
- Lecture schedule

Week	Topic
1	Linear algebra recap (least squares, eigenvalues and eigenvectors, invariant subspaces)
2	Attacks on the state estimation problem
3	Attacks on electricity markets
4	Linear system theory: state-space representation
5	Linear system theory: controllability and observability
6	LQR/LGQ control problem
7	Stealthy attacks
8	Zero dynamics
9	Replay attacks
10	DoS attacks
11	Industry perspective
12	State of the art on cyber-physical security
13	Course recap and discussions

Cyberphysical security of power systems: learning objectives

1. Understand the main challenges regarding cyber-physical attacks in networked control architectures, with a focus on power networks

2. Develop intuition about linear algebra methods, and apply them to the context of modelling successful attacks and defense mechanisms on power grids

3. Understand the main assumptions on the knowledge that the attacker possesses about the network, and how these contribute to a successful attack

4. Explore data-driven and machine learning techniques to detect attacks, as well as identify their potential to generate more complex attacks

5. Reflect on the different perspectives about the term cyber-physical attacks across different research communities and industries

Join the Vevox session

Go to **vevox.app**

Enter the session ID: **163-812-102**

Or scan the QR code

Which of the following periods would you be more likely to consider taking this course?

August 2026 — 3-week long

0%

September 2026 — 13-week long

0%

Which of the following periods would you be more likely to consider taking this course?

August 2026 — 3-week long

0%

September 2026 — 13-week long

0%

RESULTS SLIDE

PEPs and Linters

Standardizing code styles from different authors.

General

- PEP = "Python Enhancement Proposal"

- **Conventions** to standardize python code
  - Not a standard, not enforced (recommendations/suggestions)
  - Violating PEP does not break your code (not like div 0)
  - Aim is to be helpful

- Focus here:
  - PEP8: Style Guide for Python Code [2] - Code layout, naming conventions etc.
  - PEP257: Docstring Conventions [3] - How to document function, classes for user

Why do we need conventions? [2,4]

- Reading code happens more often than writing code

- "The power of consistency"
  - Improved readability (also for debugging)
    - Easier recognition of variables & their purpose
    - Easier to follow code logic (e.g. consistent spaces)
  - Prevents "Wild West" in big collaborative efforts
  - Professionalism

- However:
  - Be consistent with surrounding code more important (e. g. in-house project guidelines)

PEP257: Docstrings [3]

- In short: strings that appear at top of module or right after definition of a class, function etc. (3x quotation mark)
- Summarize purpose, input arguments and return values (+exceptions raised)
- Single-line (minimum), multi-line (detailed, PEP257)
- Can be accessed via __doc__ attribute or help() function (e.g., print(`np.array`.__doc__) or help(`np.array`)).
- Docstrings for USERS, comments for CODE MODIFIERS
- Different styles possible (e.g. **NumPy**/SciPy, google style etc.)

```python
def square(n):
    '''Takes in a number n, returns the square of n'''
    return n**2
```

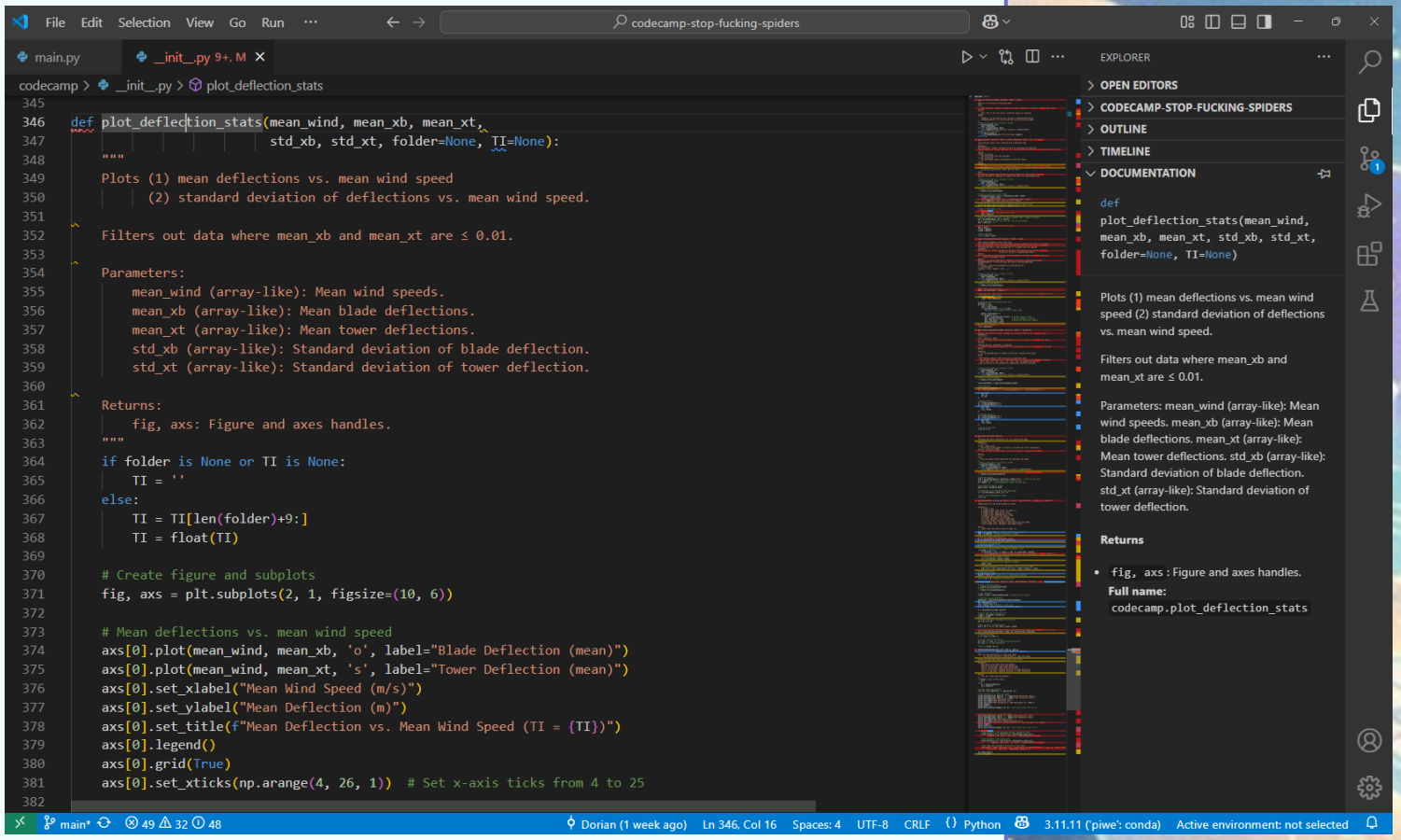
Source: https://www.programiz.com/python-programming/docstrings

```python
def add(num1, num2):
    """
    Add up two integer numbers.

    This function simply wraps the ``+`` operator, and does not
    do anything interesting, except for illustrating what
    the docstring of a very simple function looks like.

    Parameters
    ----------
    num1 : int
        First number to add.
    num2 : int
        Second number to add.

    Returns
    -------
    int
        The sum of ``num1`` and ``num2``.

    See Also
    --------
    subtract : Subtract one integer from another.

    Examples
    --------
    >>> add(2, 2)
    4
    >>> add(25, 0)
    25
    >>> add(10, -10)
    0
    """
    return num1 + num2
```

Source: https://pandas.pydata.org/docs/development/contributing_docstring.html

Why docstrings?

- Again: Easier to read and understand what's happening
- Think about the user: what does this function, class etc. do?
- Greatest strength: standardization makes post-processing by other tools easy!
  - Use Docs View extension to show your documentation in sidebar
  - Tools like Sphinx (documentation generator, [7]) can parse docstring to make beautiful documentation (example here)
- autoDocString extension to automatically create docstrings.



PEP8: Examples - a personal selection [2, 5]

Binary operators & spaces

```
# Wrong:
# operators sit far away from their operands
income = (gross_wages +
          taxable_interest +
          (dividends - qualified_dividends) -
          ira_deduction -
          student_loan_interest)
```

vs

```
# Correct:
# easy to match operators with operands
income = (gross_wages
          + taxable_interest
          + (dividends - qualified_dividends)
          - ira_deduction
          - student_loan_interest)
```

Naming conventions

Type	Public	Internal
Packages	`lower_with_under`	
Modules	`lower_with_under`	`_lower_with_under`
Classes	`CapWords`	`_CapWords`
Exceptions	`CapWords`	
Functions	`lower_with_under()`	`_lower_with_under()`
Global/Class Constants	`CAPS_WITH_UNDER`	`_CAPS_WITH_UNDER`
Global/Class Variables	`lower_with_under`	`_lower_with_under`
Instance Variables	`lower_with_under`	`_lower_with_under`
Method Names	`lower_with_under()`	`_lower_with_under()`
Function/Method Parameters	`lower_with_under`	
Local Variables	`lower_with_under`	

multiple statements per line (compound statement)

```
# Wrong:
if foo == 'blah': do_blah_thing()
do_one(); do_two(); do_three()
```

vs

```
# Correct:
if foo == 'blah':
    do_blah_thing()
do_one()
do_two()
do_three()
```

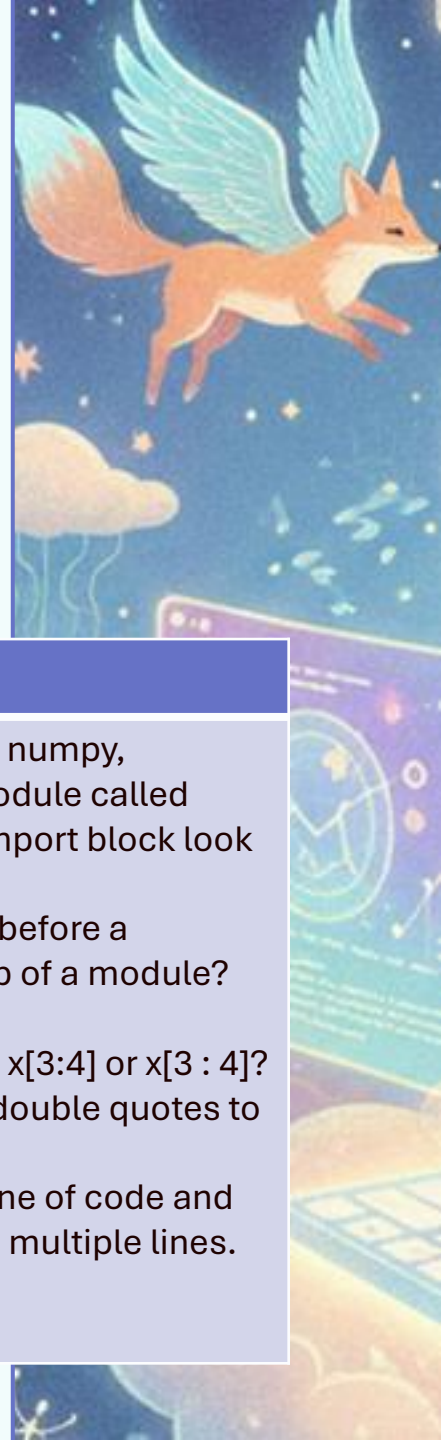
- Complete style guide: [2]

7 minutes

Small-group exercise.

- Split into groups of 2 to 3. (Online: random BORs.)
- Pick a random number from 1 to 3. That's the set of questions you will look at.
- One of you opens the PEP8 guide (https://peps.python.org/pep-0008/). Together, answer as many questions you can in 7 minutes.

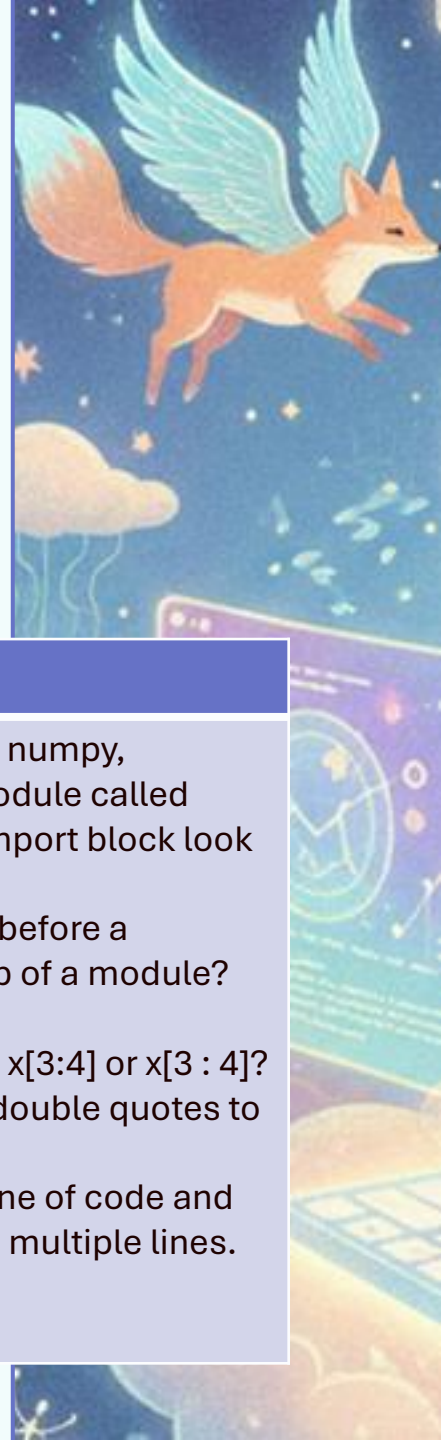
Set 1	Set 2	Set 3
1. What should you use instead of if y == None? 2. If you have an acronym in camel case, should you capitalize the first letter only or the whole acronym? 3. How should you check if a variable equals False? 4. What is important about how underscores work with variable names? What happens if I put an underscore after a variable? One before? Two before? 5. How many spaces should you use after a period in a comment?	1. What are the capitalization guidelines for classes, functions, and variables? Give examples. 2. How should you check if a path ends in .txt? 3. What is the difference between mixed case, camel case, and snake case? 4. How should you indicate block comments? 5. What are the whitespace rules around operators? Give several examples.	1. If I need to import os, sys, numpy, matplotlib, and a local module called _utils.py, what does my import block look like? 2. How many blank lines go before a function I define at the top of a module? How many after? 3. Which of these is correct: x[3:4] or x[3 : 4]? 4. Should you use single or double quotes to indicate strings? 5. Give examples of a long line of code and how you can break it onto multiple lines.



Discuss.

- Set 2. Capitalization guidelines. Previous slides: classes have capitals, no underscore.
- Set 1. To check if a variable, use “if x is None”.
- Set 3. Imports should be on different lines. But also order: built-in imports, then dependencies, then local imports.

Set 1	Set 2	Set 3
1. What should you use instead of if y == None? 2. If you have an acronym in camel case, should you capitalize the first letter only or the whole acronym? 3. How should you check if a variable equals False? 4. What is important about how underscores work with variable names? What happens if I put an underscore after a variable? One before? Two before? 5. How many spaces should you use after a period in a comment?	1. What are the capitalization guidelines for classes, functions, and variables? Give examples. 2. How should you check if a path ends in .txt? 3. What is the difference between mixed case, camel case, and snake case? 4. How should you indicate block comments? 5. What are the whitespace rules around operators? Give several examples.	1. If I need to import os, sys, numpy, matplotlib, and a local module called _utils.py, what does my import block look like? 2. How many blank lines go before a function I define at the top of a module? How many after? 3. Which of these is correct: x[3:4] or x[3 : 4]? 4. Should you use single or double quotes to indicate strings? 5. Give examples of a long line of code and how you can break it onto multiple lines.



What if I don’t know it all by heart?

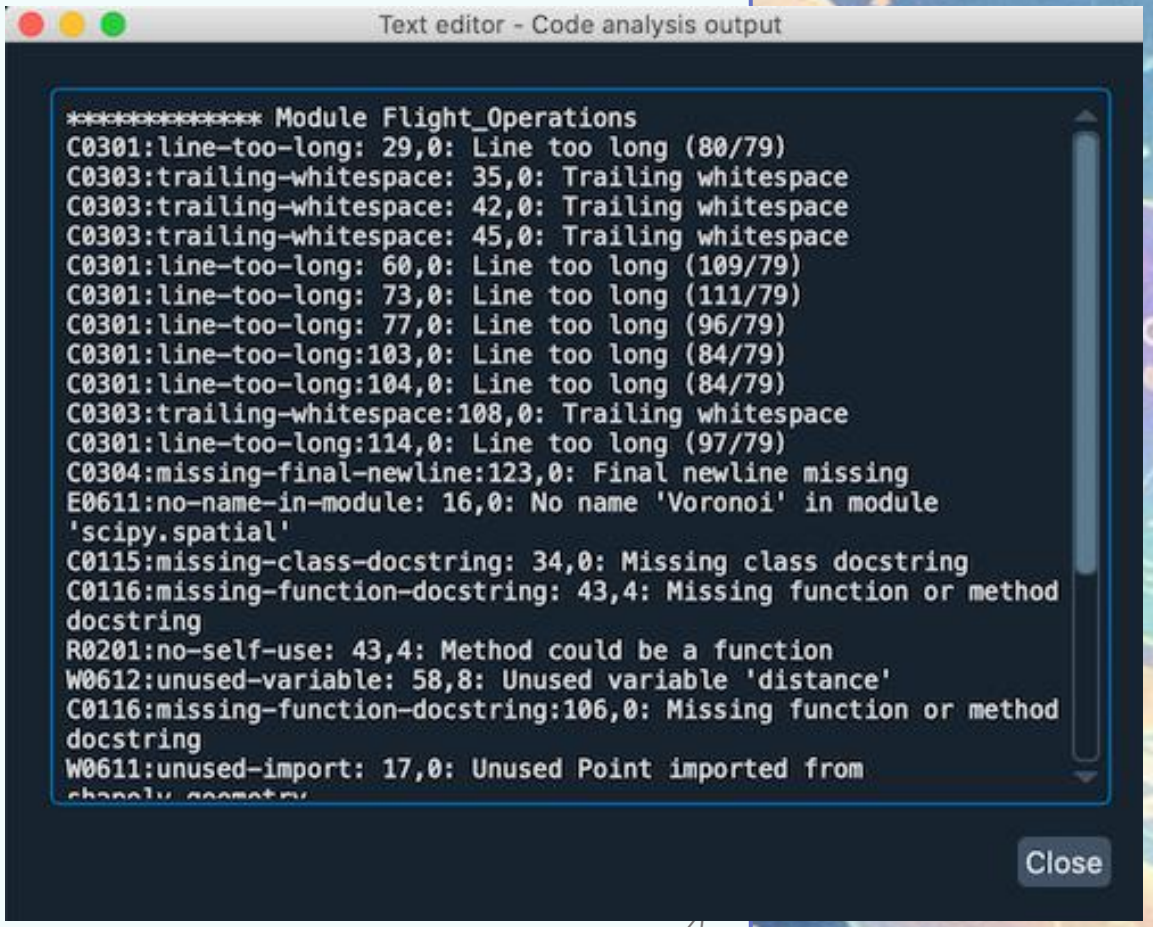
- Great. Keep space for more important things, there are tools to help you -> **Linter**
- From [9]:

> A linter is the most common and widely used of all **static code analysis tools**. Linters help developers analyze their source code to uncover potential issues, like coding errors, stylistic inconsistencies, bugs, violations of coding standards, and potential security vulnerabilities.

- **Static code analysis**: analysis of code that is done without executing it.
  - Especially important in compiled code, e.g., C.
- In short, a linter is a tool to analyze the quality of your code.

Linters in Python.

- Many different types: pylint (included in VS code, spyder, code errors and style), flake8 (VS Code), pycodestyle (mostly style checks), and more.
  - Comparison is in [11]
- Some linters only analyze PEP8, others (like pylint/flake8) do more.
- We'll use **pylint** & **flake8**. mypy for type checking.
  - Integrated in VS code extensions & on PyPI.
  - We'll see how to use it next.
- 1 step further: Auto-formatter (e.g. autopep8, black) in-place reformats your code
- More details on Python linting in [6,7].



Source: https://docs.spyder-ide.org/current/panes/pylint.html

Let’s use pylint & flake8 together.

- Can I have a volunteer team? Everyone, follow along in your own repo!

- Pylint and flake8 in VS code.
  - VS code marketplace has pylint, flake8, and mypy [12].
  - Enable both pylint and flake8. It should now run automatically on any open Python code.

- Pylint from the command line.
  - Pylint is available on PyPI. Install it with Anaconda Prompt: `pip install pylint`
  - Call pylint on a file or package: `pylint filename/packagename`

Pylint messages.

- Pylint gives different messages with codes:
  - F = fatal for pylint
  - E = probable errors
  - W = warnings
  - C = convention
  - R = refactor
  - I = information
- Full list of pylint message codes:
  - Messages overview - Pylint 4.1.0-dev0 documentation
- Not all messages are created equal.
  - E.g., is our team okay with a line being longer than 100 characters?

Disabling pylint messages.

- See Appendix for how to disable messages in a line, file, or project.
  - Two ways: specific Python comments or a settings file.
- You as a team may discuss what messages to suppress.

- Error codes you can disable with little discussion:
  - C0103 (variable not conforming to snake case). Variable names are too short

- Error codes you should think about before disabling:
  - C0301 (line too long). I often change my line length to 120.
  - R, especially R0913 (too many arguments) R0914 (too many locals) and R0917 (too many positional arguments).

How you will use this knowledge.

- Your homework this week involves two steps:

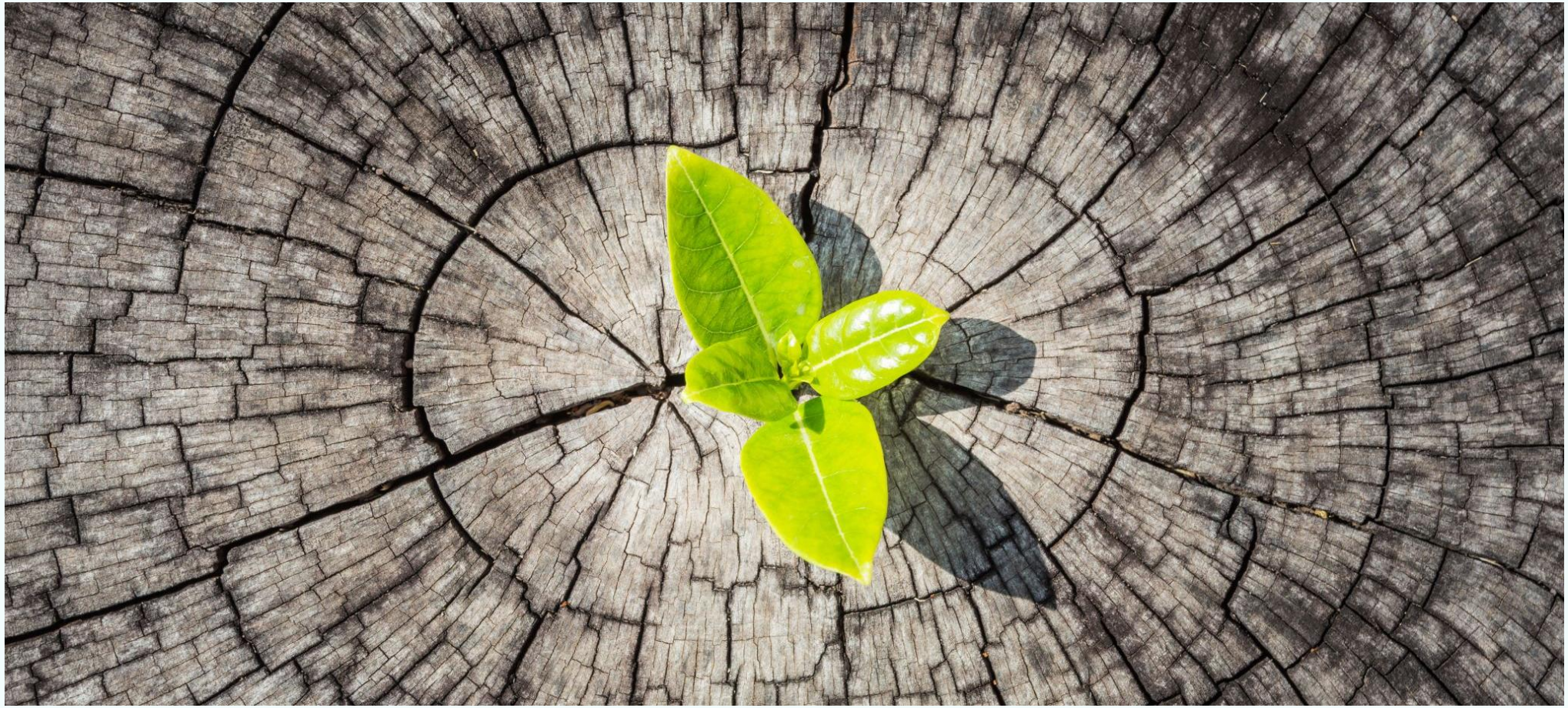
  1. Changing codecamp/ into a Python package that can be installed/used from any folder.

  2. Updating your code such that your pylint score in your package is sufficiently high.

PEP8 inspiration.

- 🎵 The PEP 8 Song 🎵 - YouTube

Break until 10.45.



Packaging.

Wrap up your code with a bow, give it as a present.

Import functions from modules or packages.

Should only have functions/classes in the package!

codecamp/ is a package!

	MODULE	PACKAGE
What it is	A single `.py` file. • E.g., `turbiefuncs.py`.	A folder with a collection of `.py` files, perhaps with subfolders. • More details later.
To import from it	Module must be in same folder as main script. • *(Or you can modify path – not recommended*)*	If not installed: • Must be in same folder as main script. If installed: • Can be imported from any folder.

Example	`code_folder/`
	`main_script.py`
	`codecamp.py, with functions:`
	• `function load_resp`
	• `function load_wsp`
	• `function simulate_turbie`
		`code_folder/`
		`main_script.py`
		`codecamp/`
		`__init__.py, with functions:`
		• `function load_resp`
		• `function load_wsp`
		• `function simulate_turbie`

Why packages are nice compared to modules.

- For just you:
  - They can be installed and thus imported anywhere on your computer*.
  - When you have many functions/classes, allows better organization because you can separate them into different files.
- **Disclaimer about modules:*
  - It's possible to also make a module "discoverable" anywhere on your computer.
  - This is done by either updating the `PYTHONPATH` environment variable (global change – don't do it!!!) or updating `sys.path` in each main script (see Appendix).
  - Jenni does not recommend. It (a) makes your code less robust to changes in folder structure and (b) makes your code less shareable due to implicit dependencies. Use (editable) packages!
- For other people:
  - Code is nicely enclosed.
  - You can upload it to PyPI, then your package can be installed via `pip install mypack` (see Appendix).

Definition of a package.

- A Python package:
  - Is a folder of .py files with functions/classes meant to be imported.
  - (Optional for Python 3.3+) Often contains a file called `__init__.py`*.
  - (Optional) Contains submodules and/or subpackages.

- *The __init__.py is not required for “namespace packages”.
  - Namespace packages are new since Python 3.3.
  - We’re going to ignore namespace packages and focus on regular packages. You can read about the namespace packages in [13, 14] (end of slide deck).

To make package usable...needs to be INSTALLED.

package name

NOT INSTALLED

```
codecamp/

    __init__.py

main.py  ← from codecamp import ...
```

- This only works because codecamp package is in same folder!

INSTALLED

```
src/

    codecamp/

        __init__.py

scripts/

  main.py  ← from codecamp import ...
```

- Now we can import from codecamp from ANY folder!

Importing from other files

- codecamp/
  - __init__.py ← simulate_turbie
  - io.py ← load_resp

- from codecamp import simulate_turbie
- from codecamp.io import load_resp

The two steps when developing.

MAKE THE PACKAGE

file structure

(EDITABLY) INSTALL THE PACKAGE

pip install

DO NOT use sys.path.append

Set-up to install a package.

- Use `pip install` (with special settings; coming later).
  - NOT sys.path.append.

- Requires a text file with installation settings: `pyproject.toml`.
  - FYI: Old setups use `setup.py` (deprecated).

- `pyproject.toml` defines important information about the package.
  - E.g., package name, dependencies, optional dependencies for developers, git repo, etc.

- Take a template toml and update it for your project.
  - Find one in this week's folder. (next slide)

See many more settings at https://packaging.python.org/en/latest/guides/writing-pyproject-toml/#a-full-example

Simple pyproject.toml

```
# An extremely simple pyproject.toml
# Many more options here: https://packaging.python.org/en/latest/guides/writing-pyproject-toml/

# How to build your package. "[S]trongly recommended" to include this,
# per the docs. We use hatchling, default on Python docs.
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

# Information on the package
[project]
name = "examplepackage"  # name of subfolder inside src/
description = "An example package"
version = "0.0.1"
authors = [
    { name = "I. M. Cool", email = "imcool@noreally.com" }
]
dependencies = [  # installed before your package
    "matplotlib",
    "numpy"
]

# (optional) Extra dependencies, e.g., for developers or power users
# editable install:  pip install -e .[dev]
[project.optional-dependencies]
dev = [
    "ipykernel",
    "pytest",
    "pytest-cov",
    "pylint"
]


[project.urls]
Repository = "https://github.com/me/spam.git"  # where your repo is hosted
```

name of package. do NOT include "src" here!

what packages need to be present for your package to run (IMPORTANT)

NOTE SCIPY IS MISSING

optional packages for developers

your git repo

Repo structure with installable package.

- Package:
  - Nested in a folder called `src/` for ALL projects. Convention defined by Python Packaging authority [18].
- Executable scripts:
  - Anywhere you want. I like putting them in a folder called `scripts/`, `code/`, or `examples/`.
- Tests:
  - In folder `tests/`.

```
data/
scripts/
  code_week3.py
  ...
  main.py
src/
  codecamp/
    __init__.py
tests/
  test_week3.py
  ...
.gitignore
LICENSE
pyproject.toml
README.md
```

Will see src/ folder in many projects

Package name is codecamp!!! Not src!

Okay...BUT HOW DO I INSTALL?

- When developing:
  - Create/activate a clean environment (instructions last week).
  - From repo folder: `pip install -e .` OR `pip install -e .[dev]`
    - Explanations on next slide.
  - Run main.py or whatever scripts you need
  - Make changes in `__init__.py` or in scripts as you need.

DO NOT use sys.path.append

A glimpse at what `pip install` does.

- Many things, which is why it can be so confusing!
- `pip install PACKNAME`
  - E.g., `pip install numpy`.
  - Looks for package PACKNAME on the online repository, PyPI.
  - If found, downloads files for that package into your Python distribution. A place you don't normally see.
- Note: in some installations, pip is not available in your terminal.
  - You can call it using alternative:

```
python -m pip install <options>
```

- `pip install .`
  - Be careful – there is a period at the end!
  - Look for `pyproject.toml` in current directory. Period = current directory.
  - If found, copies files for the package into your Python distribution.
  - Local files are now "dead".
- `pip install -e .`
  - "e" flag = editable installation.
  - Adds the package folder to `sys.path`.
  - Now you can edit the package files, and you will see updates on import.
- `pip install -e .[dev]`
  - Same as above, but also dev dependencies in `pyproject.toml`.

Summary of most common pip commands.

`pip install PACKNAME`	← looks for package on PyPI, installs dependencies, downloads package files from internet to Python folder.
`pip install .`	← finds package settings in `pyproject.toml` in current folder, installs dependencies, copies package files from local folder to Python folder.
`pip install -e .`	← finds package settings in `pyproject.toml` in current folder, installs dependencies, adds folder to Python path

Note pip can also directly install from a git repo if git is installed, and much more. See pip docs for more details [16].

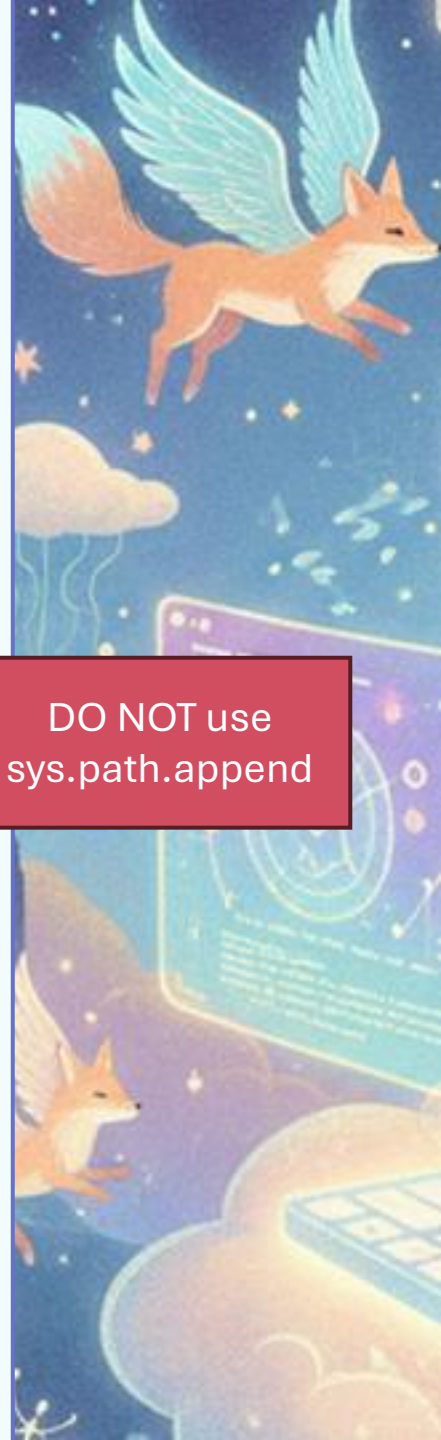
How you will use this knowledge.

- Your homework this week involves two steps:

1. Changing codecamp/ into a Python package that can be installed/used from any folder.

2. Updating your code such that your pylint score in your package is sufficiently high.

DO NOT use sys.path.append



Questions?

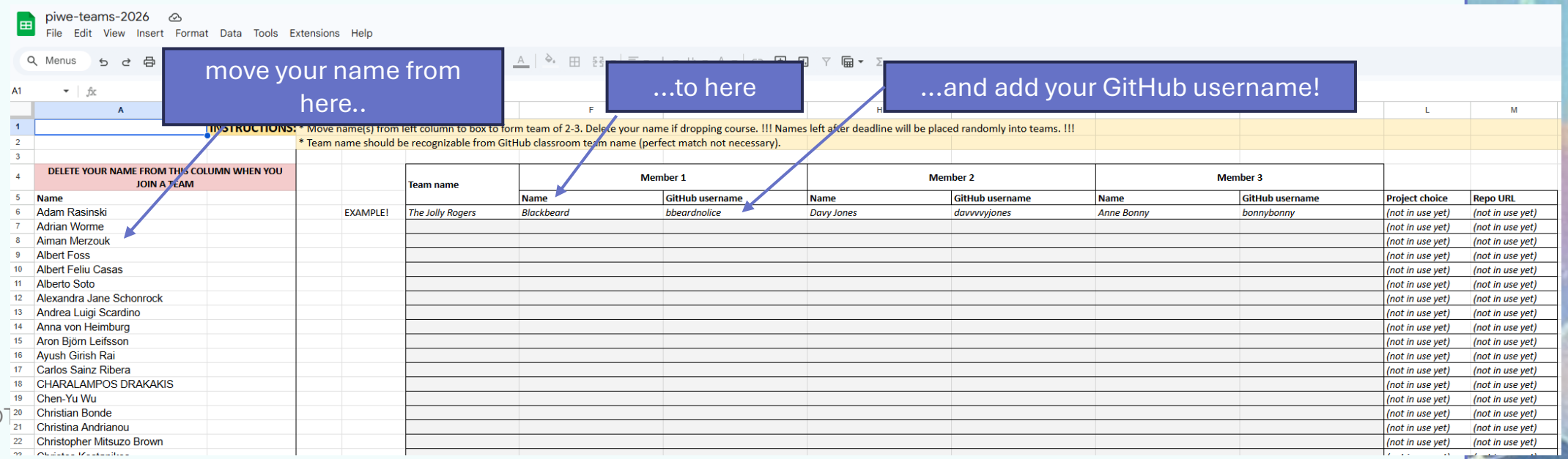
Homework for this week.

Clean up your codecamp code.

Optional: form final project teams

- By Sunday, March 29 at 23:59, form a team for final project and sign up your team. Fill out all info in a row AND <u>delete your name from left column</u>.
- SIGN-UP SHEET FORMAT HAS CHANGED. See below.
    1. Make up a team name.
    2. Copy your name from left colum AND add your github username. BE ACCURATE WITH GITHUB USERNAME. Otherwise, repo access problems.

Deadline is after next week, thus “optional”.



Last homework with codecamp teams.

Details on GitHub

- Two feature branches, completed serially. In short:
  1. Restructure your codecamp project to have an installable package. Test package installation using an environment.
  2. Update package to have pylint score of 7+.

- Optional:
  - Fix issues from CodeCamp project.
  - Update your scripts/main to also have pylint score of 7+.
  - Upload your package to PyPI so it can be installed easily, like numpy.
  - Review project descriptions from last year, form a team for the final project.

Any questions?

Appendix

References.

1. Summary of PEP8, https://vuyisile.com/a-summary-of-pep-8-style-guide-for-python-code-and-pep-257-docstring-conventions/
2. PEP8 – Style Guide for Python Code, official documentation, https://peps.python.org/pep-0008/
3. PEP257 – Docstring Conventions, official documentation, https://peps.python.org/pep-0257/
4. Jasmine Finer, How to Write Beautiful Python Code With PEP 8, https://realpython.com/python-pep8/
5. The Naming Convention Foundation, python Naming Convention, https://github.com/naming-convention/naming-convention-guides/tree/master/python
6. Parisa Gregg & Myles Mitchell, Stylising your Python code: An introduction to linting and formatting https://www.jumpingrivers.com/blog/python-linting-guide/
7. Essential Python Tools, https://books.agiliq.com/projects/essential-python-tools/en/latest/linters.html
8. CodeCamp final project Final project — CodeCamp 2023 documentation (dtu.dk)
9. What is a Linter? What Are Linters? (+ Why Your Team Should Use Them) (codacy.com)
10. A Deep Dive into Static Code Analysis Tools A Deep Dive into Static Code Analysis Tools (codacy.com)
11. Comparison of Python linters Linters and formatters — Essential Python Tools 3.7 documentation (agiliq.com)
12. Linting in VS code Linting Python in Visual Studio Code

References continued.

13. Tutorial on packages, implicit namespace package Python Modules and Packages – An Introduction – Real Python

14. Namespace package and what they're good for What's a Python Namespace Package, and What's It For? – Real Python

15. Pyproject.toml Writing your pyproject.toml - Python Packaging User Guide

16. Pip install docs pip install - pip documentation v24.0 (pypa.io)

17. Generating distribution archives and uploading them to TestPyPI Packaging Python Projects - Python Packaging User Guide

18. Python Packaging User Guide src layout vs flat layout src layout vs flat layout - Python Packaging User Guide

How to upload your package to PyPI.

- This will allow a user to install your package with `pip install <your_package_name>`, without needing git.
- Excellent explanation and steps here on the Python Packaging User Guide:
  - Packaging Python Projects - Python Packaging User Guide

Controlling pylint messages

Control over disabling messages.

- Detailed docs here: [Messages control - Pylint 4.1.0-dev0 documentation]

- Paraphrased from docs: pylint accepts the following values:
  - a symbolic message: `no-member`, `undefined-variable` etc.
  - a numerical ID: `E1101`, `E1102` etc.
  - The name of the group of checks. Can grab those with pylint --list-groups.
  - Corresponding category of the checks. E.g., `C`, `R`, etc.
  - All the checks with `all`

Option 1: control messages in a file.

- More details: [Messages control - Pylint 4.1.0-dev0 documentation]
- Disable warning on a line:

```
a, b = ... # pylint: disable=unbalanced-tuple-unpacking
```

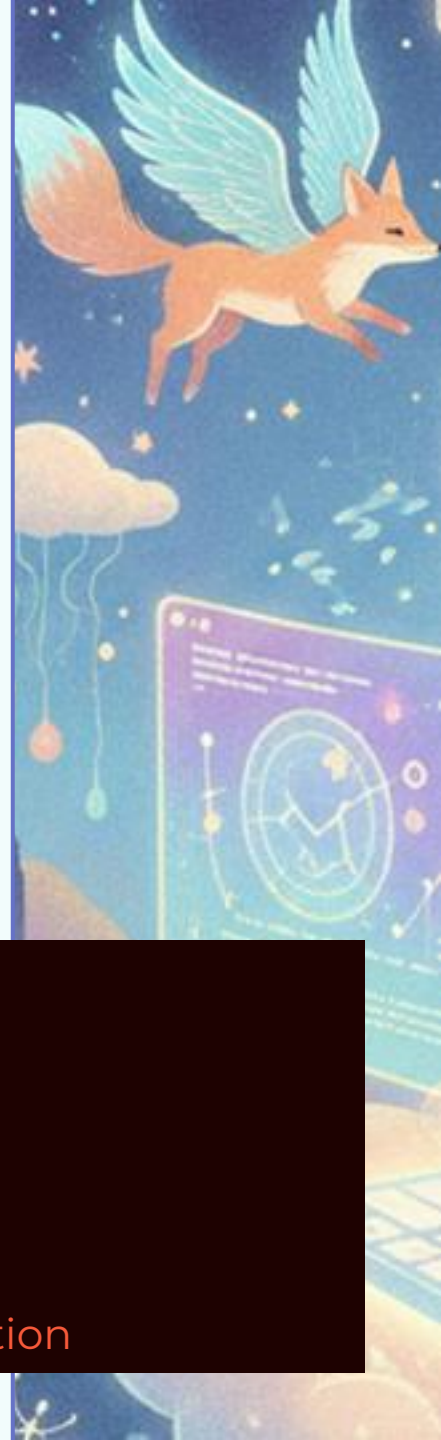
- Disable warning on next line:

```
# pylint: disable-next=unbalanced-tuple-unpacking
a, b = ...
```

- Disable warning in module:

```
“““Module docstring goes here”””
# python: disable=C0103
(rest of code)
```

Option 2: control messages in a project or globally.

- Can store “global” pylint settings in file with specific name.
  - E.g., named `pylintrc` or `.pylintrc`.
  - “RC” stands for “run commands”, a.k.a., settings initiated when pylint starts.
- Place .pylintrc in root of your repo to cover all code in repo.
  - Extension will auto-load in the folder that is opened in VS Code.
- Example file contents:
  - See more on pylint/pylintrc at main

```
[MESSAGES CONTROL]
disable=C0103  # invalid name

[FORMAT]
max-line-length=120  # extend line length

[DESIGN]
max-args = 10   # don’t flag if 11+ args in a function
```